CTL

Car Technical Log

Mobile Application — iOS & Android

Technical Documentation

Version 1.0.0 · March 2026 · Build 2

PLATFORM	iOS (TestFlight) & Android (Internal Testing)
BACKEND	Node.js 24 · Express 5 · PostgreSQL 17
MOBILE	Expo SDK 54 · React Native · Expo Router
AUTH	JWT + bcryptjs · Face ID / Touch ID / Fingerprint
NOTIFICATIONS	expo-notifications 0.32 · Local push (weekly + event reminders)

Table of Contents

1.	Product Overview	3
2.	Architecture & Technology Stack	4
3.	Application Screens & Features	5
4.	Authentication & Security	8
5.	Data Model	9
6.	REST API Reference	11
7.	Push Notification System	14
8.	Theme System (Dark Mode)	15
9.	Build & Deployment	16
10.	Known Limitations & Roadmap	17

1 PRODUCT OVERVIEW

What is CTL?

CTL (Car Technical Log) is an aviation-style digital logbook for car owners and fleet managers. Inspired by the rigorous maintenance records kept in commercial aviation, CTL brings the same discipline to personal vehicle ownership — enabling users to track every maintenance event, parts replacement, fuel fill-up, insurance policy, and dealership visit across one or more vehicles.

The app targets drivers who want complete visibility into their vehicle's full history, upcoming service schedule, and total cost of ownership. It generates comprehensive per-vehicle reports and proactively alerts users to upcoming service events via push notifications.

Ø=ÜË

Complete History

Every service, part, inspection & repair in one place

Ø=Ý

Smart Reminders

Push notifications before service dates & due mileage

Ø=Þ—

Multi-Vehicle

Manage your entire fleet from a single account

Ø=Ü°

Cost Tracking

Full spend breakdown per vehicle and category

Ø=ÜÊ

Reports

Shareable per-car logbook reports in PDF format

Ø<ß

Dark Mode

Full light / dark / system theme with instant switching

Distribution Status

iOS TestFlight

Android Testing

Version 1.0.0

Build 2

2 ARCHITECTURE & TECHNOLOGY STACK

System Architecture

CTL follows a standard client-server architecture. The Expo mobile app communicates with a dedicated Express REST API over HTTPS. All persistent data lives in a PostgreSQL database managed via Drizzle ORM. JWT tokens are stored securely on-device using the iOS Keychain and Android Keystore.



Monorepo Structure

```
workspace/
% % %  artifacts/
%    % % %  api-server/        !• Express REST API (TypeScript + Drizzle)
%    % % %  car-logbook/       !• Expo Mobile App (React Native + Router)
% % %  lib/
%    % % %  db/                !• Drizzle ORM schema + PostgreSQL connection
%    % % %  api-spec/          !• OpenAPI YAML specification
%    % % %  api-client-react/  !• Auto-generated React Query hooks (Orval)
% % %  pnpm-workspace.yaml    !• Workspace package manager config
```

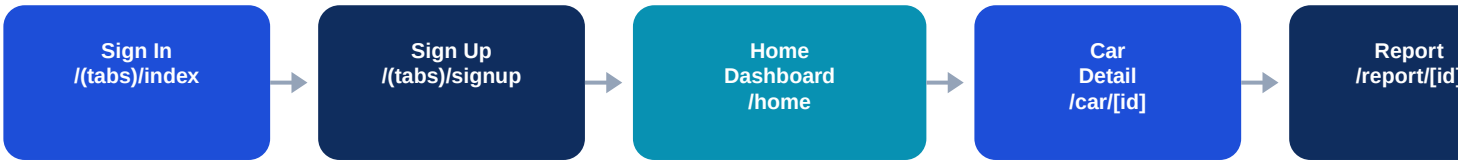
Technology Stack

Layer	Technology	Version	Purpose
Mobile App	Expo / React Native	SDK 54	Cross-platform iOS & Android
Navigation	Expo Router	6.0.x	File-based routing (Next.js style)
Server State	TanStack Query	5.x	Data fetching, caching, sync
API Server	Express	5.x	REST API framework
Language	TypeScript	5.9	End-to-end type safety
Database	PostgreSQL	17.x	Primary relational data store
ORM	Drizzle ORM	0.39.x	Type-safe schema + queries
Auth — Server	JWT + bcryptjs	—	Token auth + password hashing
Auth — Client	expo-secure-store	15.x	iOS Keychain / Android Keystore
Biometrics	expo-local-authentication	17.x	Face ID / Touch ID / Fingerprint
Notifications	expo-notifications	0.32.x	Local push notifications
Build / CI	EAS (Expo App Services)	—	Native build + TestFlight submit
Package Mgr	pnpm workspaces	10.x	Monorepo dependency management

3 APPLICATION SCREENS & FEATURES

Navigation Overview

CTL uses Expo Router's file-based routing. The bottom tab bar is always visible. Once authenticated, users land on the Home Dashboard — a command centre for all vehicle actions.



Screen Directory

Screen	Route	Description
Home Dashboard	/home	Quick-action hub + upcoming events overview
My Fleet	/fleet	All vehicles list with quick stats
Car Detail	/car/[id]	Tabbed: Maintenance · Parts · Insurance · Dealerships · Fuel
Add Car	/car/add	Modal form — add a new vehicle with photo
Edit Car	/car/edit	Edit vehicle details and odometer
Upcoming Events	/maintenance/upcoming	Cross-fleet service schedule with urgency colours
Add Maintenance	/maintenance/add	Log a new maintenance record
Add Parts	(via car detail)	Log a parts replacement
Add Malfunction	/malfunction/add	Log a fault code or warning light
Malfunction Log	/malfunction/log	View all malfunctions fleet-wide
Add Inspection	/inspection/add	Log a vehicle inspection (MOT etc.)
Add Renewal	/renewal/add	Log an insurance renewal
Repair History	/repair-history	Full chronological repair timeline
History	/history	Complete event history
Log Event	/event/add	Quick-log any event type
Report	/report/[id]	Full logbook report — cost breakdown + timeline
Settings	/settings	Theme · notifications · account · sign out

Feature Set

Vehicle Management

- Make, model, year, color, VIN, license plate
- Current mileage and photo
- Unlimited vehicles per account
- Edit or delete at any time

Maintenance Tracking

- Service type, date, mileage, cost, shop
- Next due date and/or next due mileage
- Upcoming events dashboard
- Urgency color-coding (see below)

Parts Tracking

- Part name, brand, category, part number
- Cost, supplier, warranty expiry
- OEM and aftermarket parts support

Insurance Management

- Provider, policy number, type
- Premium, start/end date
- Coverage notes and renewal tracking

Dealership Visits

- Shop name, location, purpose
- Cost, contact info, 1–5 star rating
- Track which shop did what work

Fuel Log

- Litres, cost per litre, total cost
- Mileage, fuel type, station name
- Economy trend tracking

Urgency Colour System

Upcoming events are colour-coded by days remaining, mirroring the aviation logbook concept of urgency:

Ø=Ý4

OVERDUE

Past the due date

Ø=ßà

URGENT

Due within 7 days

Ø=ßá

APPROACHING

Due within 30 days

Ø=ßâ

SCHEDULED

More than 30 days

4 AUTHENTICATION & SECURITY

Authentication Flow

CTL uses JWT-based authentication. Passwords are never stored in plain text — they are hashed with bcryptjs (12 salt rounds) before storage. Tokens are stored in expo-secure-store which maps to the iOS Keychain and Android Keystore.



Biometric Sign-In

- Available on devices with enrolled Face ID, Touch ID, or fingerprint
- Stored JWT is retrieved from secure storage and validated via OS biometric prompt
- No credentials are transmitted — only the cached token is used
- User preference persisted under key: ctl_biometric_enabled

Secure Storage Keys

Key	Stored Value	Backend
ctl_auth_token	JWT bearer token (7-day expiry)	iOS Keychain / Android Keystore
ctl_auth_user	Serialised user object (JSON)	iOS Keychain / Android Keystore
ctl_biometric_enabled	User biometric preference (bool)	iOS Keychain / Android Keystore
ctl_theme_preference	light / dark / system	iOS Keychain / Android Keystore

Security Posture

- All API endpoints (except register/login) require a valid Bearer token
- Server validates JWT signature and expiry on every request
- Tokens expire after 7 days; users must re-authenticate after expiry
- Account deletion cascades — removes all vehicles and all records permanently
- HTTPS enforced in production via TLS

& Password Reset: Reset tokens are currently logged to the API server console only. Email delivery (via Resend / SendGrid) is planned for the next release.

5 DATA MODEL

Entity Relationship Overview

All tables use integer serial primary keys. Child records cascade-delete when a parent is removed.

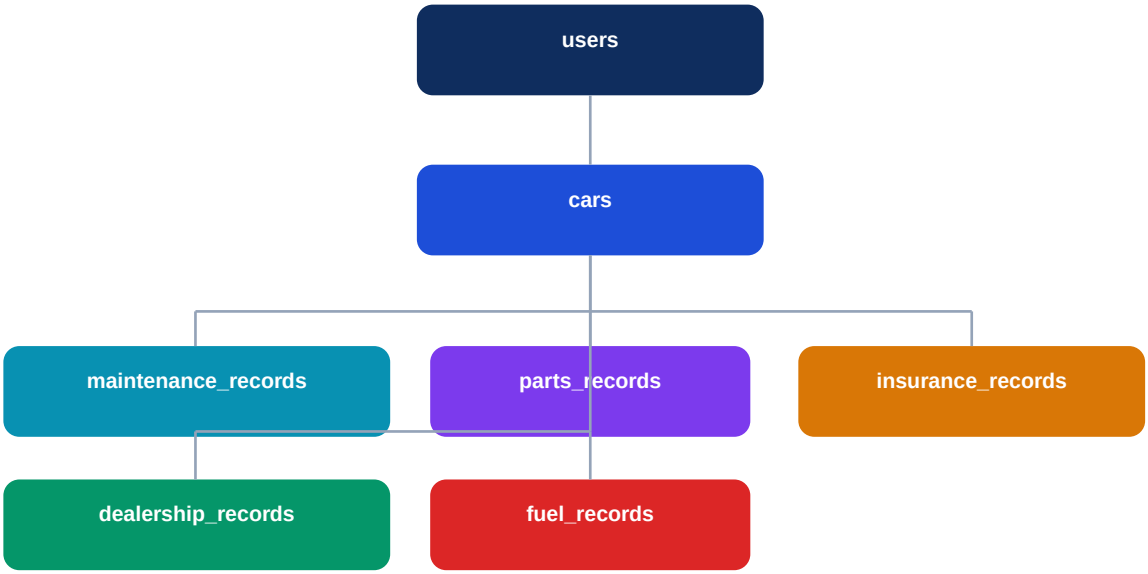


Table Schemas

users

Column	Type	Constraints	Description
id	serial	PRIMARY KEY	Auto-increment identifier
name	varchar(255)	NOT NULL	Display name
email	varchar(255)	UNIQUE NOT NULL	Login email address
password_hash	varchar(255)	NOT NULL	bcryptjs hash (12 rounds)
created_at	timestamp	DEFAULT now()	Account creation timestamp

cars

Column	Type	Constraints	Description
id	serial	PRIMARY KEY	Auto-increment identifier
user_id	integer	FK !’ users(id)	Owner of this vehicle
make	varchar(100)	NOT NULL	Manufacturer (e.g. Toyota)
model	varchar(100)	NOT NULL	Model name (e.g. Camry)
year	integer	NOT NULL	Model year
color	varchar(50)	nullable	Vehicle colour
vin	varchar(50)	nullable	Vehicle identification number
license_plate	varchar(50)	nullable	Registration plate
mileage	integer	nullable	Current odometer reading
photo_url	text	nullable	Vehicle photo URL
nickname	varchar(100)	nullable	Friendly name / alias
created_at	timestamp	DEFAULT now()	Record creation timestamp

maintenance_records

Column	Type	Notes
id	serial PK	Auto-increment identifier
car_id	integer FK	References cars(id) — cascade delete
type	varchar(100)	e.g. Oil Change, MOT, Tyre Rotation
description	text	Detailed notes (optional)
date	date	Date the service was performed
mileage	integer	Odometer at time of service
cost	numeric(10,2)	Cost of the service
shop	varchar(200)	Name of the service centre
next_due_date	date	When next service is due (drives notifications)
next_due_mileage	integer	Mileage at which next service is due

parts_records

Column	Type	Notes
id	serial PK	Auto-increment identifier
car_id	integer FK	References cars(id)
name	varchar(200)	Part description
brand	varchar(100)	Manufacturer brand
category	varchar(100)	e.g. Brakes, Tyres, Filters, Belts
part_number	varchar(100)	OEM or aftermarket part number
cost	numeric(10,2)	Purchase cost
date_replaced	date	Date installed on vehicle
supplier	varchar(200)	Where purchased
warranty_date	date	Warranty expiry date (optional)

Other Tables

- insurance_records — provider, policy_number, type, premium, start_date, end_date, coverage_notes
- dealership_records — name, location, purpose, date, cost, contact, rating (1–5), notes
- fuel_records — date, litres, cost_per_litre, total_cost, mileage, fuel_type, station
- malfunctions — fault_code, description, warning_light, date, resolved, resolution_cost
- inspections — type, date, result (pass/fail), mileage, inspector, next_due_date, notes

6 REST API REFERENCE

Base URL & Auth Header

Development	http://localhost:8080
Production	https://[deployed-domain].replit.app
Auth header	Authorization: Bearer <jwt_token>
Content-Type	application/json

> All endpoints except POST /auth/register and POST /auth/login require a valid Bearer token in the Authorization header. Missing or expired tokens return HTTP 401 Unauthorized.

Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/auth/register	No	Create a new user account
POST	/auth/login	No	Login — returns JWT
GET	/auth/me	Yes	Get current authenticated user profile
DELETE	/auth/account	Yes	Permanently delete account and all data

Vehicles

Method	Endpoint	Auth	Description
GET	/cars	Yes	List all vehicles for authenticated user
POST	/cars	Yes	Create a new vehicle
GET	/cars/:carId	Yes	Get single vehicle by ID
PUT	/cars/:carId	Yes	Update vehicle details
DELETE	/cars/:carId	Yes	Delete vehicle and all associated records

Maintenance

Method	Endpoint	Auth	Description
GET	/maintenance/upcoming	Yes	All upcoming events across entire fleet
GET	/maintenance	Yes	All maintenance records (fleet-wide)
GET	/cars/:id/maintenance	Yes	Maintenance records for one vehicle
POST	/cars/:id/maintenance	Yes	Log a new maintenance record
PUT	/cars/:id/maintenance/:rid	Yes	Update maintenance record
DELETE	/cars/:id/maintenance/:rid	Yes	Delete maintenance record

Parts / Insurance / Dealerships / Fuel

Method	Endpoint	Auth	Description
GET	/cars/:id/parts	Yes	Parts records for one vehicle
POST	/cars/:id/parts	Yes	Log a new parts record
PUT	/cars/:id/parts/:rid	Yes	Update parts record
DELETE	/cars/:id/parts/:rid	Yes	Delete parts record
GET	/cars/:id/insurance	Yes	Insurance records for one vehicle
POST	/cars/:id/insurance	Yes	Log a new insurance policy
PUT	/cars/:id/insurance/:rid	Yes	Update insurance record
DELETE	/cars/:id/insurance/:rid	Yes	Delete insurance record
GET	/cars/:id/dealerships	Yes	Dealership visits for one vehicle
POST	/cars/:id/dealerships	Yes	Log a new dealership visit
PUT	/cars/:id/dealerships/:rid	Yes	Update dealership record
DELETE	/cars/:id/dealerships/:rid	Yes	Delete dealership record
GET	/cars/:id/fuel	Yes	Fuel records for one vehicle
POST	/cars/:id/fuel	Yes	Log a new fuel fill-up
PUT	/cars/:id/fuel/:rid	Yes	Update fuel record
DELETE	/cars/:id/fuel/:rid	Yes	Delete fuel record

Malfunctions / Inspections / Reports

Method	Endpoint	Auth	Description
GET	/malfunctions	Yes	All malfunctions fleet-wide
GET	/cars/:id/malfunctions	Yes	Malfunctions for one vehicle
POST	/cars/:id/malfunctions	Yes	Log a new malfunction / fault
DELETE	/cars/:id/malfunctions/:rid	Yes	Delete a malfunction record
GET	/inspections	Yes	All inspections fleet-wide
GET	/cars/:id/inspections	Yes	Inspections for one vehicle
POST	/cars/:id/inspections	Yes	Log a new inspection
DELETE	/cars/:id/inspections/:rid	Yes	Delete an inspection record
POST	/cars/:id/events/complete	Yes	Log a completed event
GET	/cars/:id/events	Yes	Event history for a vehicle
GET	/cars/:id/report	Yes	Full logbook report data

7 PUSH NOTIFICATION SYSTEM

Overview

CTL uses expo-notifications (v0.32.x) to deliver local push notifications — no third-party push server required. Notifications are scheduled on-device each time the app comes to the foreground, based on upcoming events fetched from the API.

Ø=ÜÅ

Weekly Digest

Every Sunday at 9 AM
Lists top 5 events
sorted by urgency

Ø=ßá

30-Day Reminder

30 days before due date
Fired at 9 AM local time
For each upcoming event

Ø=ßà

7-Day Reminder

7 days before due date
Fired at 9 AM local time
For each upcoming event

Ø=Ý4

1-Day Reminder

Day before due date
Fired at 9 AM local time
"Due tomorrow" alert

Weekly Digest — Example

Ø=ÜÅ CTL Weekly — 3 upcoming events

Ø=Ý4 Oil Change · 2021 Toyota Camry · 3d overdue
Ø=ßà Tyre Rotation · 2020 BMW 3 Series · in 5 days
Ø=ßá MOT Test · 2021 Toyota Camry · in 22 days

How Scheduling Works

- On login and every app foreground: fetch /maintenance/upcoming from API
- Cancel ALL previously scheduled notifications (cancelAllScheduledNotificationsAsync)
- Schedule individual reminders for events with future 30d / 7d / 1d thresholds
- Schedule the weekly digest for next Sunday at 9:00 AM local time
- If all caught up: schedule "' All caught up!" weekly notification instead
- Web platform is excluded — notifications only on native iOS and Android

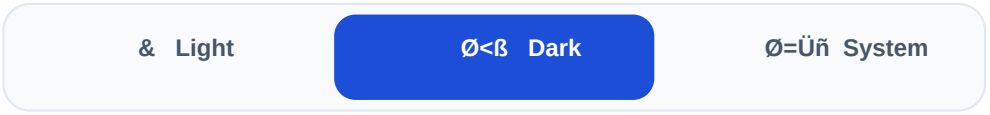
Required Permissions

Platform	Permission	Purpose
iOS	NSUserNotificationUsageDescription	System permission prompt shown on first open
Android	POST_NOTIFICATIONS	Required on Android 13+
Android	SCHEDULE_EXACT_ALARM	Precise alarm scheduling
Android	RECEIVE_BOOT_COMPLETED	Persist notifications across device restarts

8 THEME SYSTEM (DARK MODE)

Overview

CTL ships a full Light / Dark / System theme engine. The user's preference is persisted in expo-secure-store (iOS Keychain / Android Keystore) and applied instantly across all 42 screens and components without requiring an app restart.



!• Segmented control in Settings screen (haptic feedback on tap)

Theme Architecture

- ThemeContext.tsx — React context providing useColors() and useTheme() hooks
- colors.ts — Defines both light and dark ColorScheme with semantic token names
- useColors() — Returns the active ColorScheme (light or dark) based on preference + OS
- useTheme() — Exposes { preference, setPreference, resolvedTheme }
- All 42 screens/components call useColors() — zero hardcoded colour values

Semantic Colour Tokens

Token	Light	Dark	Semantic Use
background	#F1F5F9	#0F172A	Screen backgrounds
card	#FFFFFF	#1E293B	Cards, panels, surfaces
text	#0F172A	#F1F5F9	Primary body text
textSecondary	#475569	#94A3B8	Subtitles, metadata
tint	#1D4ED8	#3B82F6	Interactive / brand colour
border	#CBD5E1	#334155	Dividers and outlines
danger	#DC2626	#EF4444	Errors, destructive actions
warning	#D97706	#F59E0B	Warning states
success	#059669	#10B981	Success, completed states
shadow	#64748B	#000000	Card drop shadows

9 BUILD & DEPLOYMENT

EAS Build Configuration

EAS Account	mkashour
iOS Bundle ID	com.ctl.carlogbook
Android Package	com.ctl.carlogbook
iOS Build Number	2 (current TestFlight)
App Version	1.0.0
Build Profile	preview
EAS Project ID	2984157c-0a32-485b-8b48-1048d72a1520

Build Commands

```
# iOS build
EXPO_TOKEN=<token> npx eas-cli build --platform ios --profile preview --non-interactive --no-wait
```

```
# Android build
EXPO_TOKEN=<token> npx eas-cli build --platform android --profile preview --non-interactive --no-wait
```

```
# Submit to TestFlight (from Mac with Xcode credentials)
npx eas-cli submit --platform ios --latest
```

```
# Database schema sync
pnpm --filter @workspace/db run push
```

Distribution Status

Platform	Channel	Status	Current Build	Notes
iOS	TestFlight	In Review	Build 2 (v1.0.0)	Submitted after build number fix
Android	Internal Testing	Active	Preview APK	Available to internal testers

Environment & Infrastructure

- API Server: Replit hosted (dev) — promote to Replit Deployment for always-on production
- Database: Replit-managed PostgreSQL — persistent storage, accessible via DATABASE_URL env var
- Mobile builds: EAS cloud build service — no local Xcode or Android Studio required
- Expo SDK: 54 — React Native 0.81.5, New Architecture enabled

> Recommended next step: Deploy the API server to a Replit Deployment (always-on) to eliminate cold-start latency for testers. The current dev server may sleep after periods of inactivity.

10 KNOWN LIMITATIONS & ROADMAP

Current Limitations

Area	Limitation	Priority
Password Reset	Email delivery not yet wired — token logged to console only	High
API Uptime	API server may sleep on inactivity (not yet always-on deploy)	High
Notifications	Digest content reflects event state at scheduling time, not at firing time	Medium
Notifications	Full notification features require native build (not Expo Go)	Medium
Distance Unit	Kilometres hardcoded; miles toggle is UI-only, not functional	Low
Profile Edit	Edit Profile screen not yet implemented (Coming Soon badge)	Low
In-App Feedback	Feedback form not yet built	Low
Privacy Policy	Privacy policy page not yet implemented	Low
CTL Fleet Tier	Multi-seat fleet management upgrade — design phase only	Low

Roadmap

Immediate (Next Build)

- Email password reset (Resend / SendGrid)
- Always-on API deployment
- App Store screenshots

Short Term (1–2 months)

- Onboarding flow
- Edit Profile screen
- km !" miles toggle
- In-app feedback form

Medium Term (3–6 months)

- Cloud push notifications
- Fuel economy charts
- Full-text record search
- Cloud photo storage

Long Term (CTL Fleet)

- Multi-user / team accounts
- Fleet analytics dashboard
- OBD-II scanner integration
- Mechanic portal